

La flemme olympique

Table des matières

1	Bienvenue aux Flemmes Olympiques !	1
2	Description du problème	2
3	Partition parfaite des entiers de 1 à n	5
4	Algorithme glouton	6
5	Algorithme glouton répété	8
6	Programmation dynamique	10
7	Tester et Générer	14
8	Problèmes connexes	14

1 Bienvenue aux Flemmes Olympiques !

Votre mission : créer le résumé parfait des Jeux Olympiques, en combinant des séquences de différents sports. Certaines sont courtes, d'autres longues, certaines spectaculaires, d'autres plus calmes ... et votre objectif est de tout organiser pour que chaque segment ait exactement la bonne durée.

Pourquoi cette précision ? Les diffusions reposent sur des serveurs capricieux et des automatismes stricts. Une vidéo trop longue ou trop courte peut bloquer tout le système, interrompre d'autres diffusions, et faire grincer des dents dans les salles de contrôle.

Au début, vous tentez d'organiser les séquences à la main, en espérant gagner du temps. Mauvaise idée. Les vidéos s'entrechoquent, certaines séquences débordent, d'autres restent à moitié inutilisées ... et vous commencez à désespérer.

Mais pas de panique ! Heureusement, Richard Ernest Bellman, maître des algorithmes, peut vous aider à trouver une partition parfaite lorsque c'est possible, et la plupart du temps dans un temps raisonnable, sauf dans les pires cas.

Et pour vous guider, l'esprit d'Alan Jay Perlis, pionnier de la programmation et des langages informatiques, est là pour distiller ses conseils et réflexions sur les algorithmes ... à travers ses fameuses épigrammes [6].

Sa première leçon : simplifiez le problème. Considérez chaque séquence comme un objet réduit à sa seule durée et préparez-vous à appliquer des algorithmes plus ou moins retors selon les situations.

Alan vous avertit cependant que l'efficacité et les gains de temps n'arriveront qu'après un sérieux effort pour comprendre le problème et dompter les algorithmes. Mais une fois ce coup de maître acquis, vous pourrez enfin profiter de votre flemme olympique ... sans catastrophe vidéo.

Prêts à devenir des champions de la flemme en domptant la puissance des algorithmes ?

On croit comprendre en apprenant, on est plus confiant en écrivant, encore plus en enseignant. ... mais on en est vraiment sûr en programmant.

– Alan J. Perlis

2 Description du problème

Définition 2.1 – Sac d'entiers

Un *sac d'entiers* est une collection d'entiers naturels dans laquelle un même nombre peut apparaître plusieurs fois.

Par exemple, le sac

$$S = \{1, 2, 2, 5\}$$

contient quatre entiers et le nombre 2 apparaît deux fois.

Plus généralement, un sac S de n entiers naturels peut être noté

$$S = \{s_1, s_2, \dots, s_n\}$$

où chaque s_i est un entier strictement positif.

Définition 2.2 – Partition d'un sac

Une *partition* d'un sac S consiste à séparer les éléments de S en deux sacs non vides S_1 et S_2 . Chaque élément du sac initial doit apparaître dans *exactement un* des deux sacs.

Autrement dit :

- S_1 et S_2 sont non vides ;
- S_1 et S_2 sont disjoints (aucun entier n'apparaît dans les deux sacs) ;
- S_1 et S_2 recouvrent S : tous les éléments de S apparaissent dans S_1 ou S_2 .

Exemple – Partition d'un ensemble

Soit un sac $S = \{1, 2, 3, 4, 5\}$.

- Les sacs S_1 et S_2 forment une partition de S .
 - $S_1 = \{1\}$ et $S_2 = \{2, 3, 4, 5\}$
 - $S_1 = \{2, 4\}$ et $S_2 = \{1, 3, 5\}$
- Les sacs S_1 et S_2 ne forment pas une partition de S .
 - $S_1 = \{1, 2, 3, 4, 5\}$ et $S_2 = \emptyset$, car S_2 est vide.
 - $S_1 = \{1, 2, 3\}$ et $S_2 = \{3, 4, 5\}$, car le nombre 3 apparaît dans les deux sacs.
 - $S_1 = \{1, 2\}$ et $S_2 = \{4, 5\}$, car le nombre 3 n'apparaît dans aucun des deux sacs.

Définition 2.3 – Partition parfaite d'un sac pair

Un sac d'entiers S est dit *pair* si la somme de tous ses entiers est un nombre pair.

Une *partition parfaite* d'un sac pair est une partition (S_1, S_2) telle que la somme des entiers de S_1 est égale à la somme des entiers de S_2 .

Exemple – Partition parfaite d'un sac pair

$S = \{1, 2, 3, 4\}$ est un sac pair.

- $S_1 = \{1, 3\}$ et $S_2 = \{2, 4\}$ ne forment pas une partition parfaite, car $1 + 3 \neq 2 + 4$.
- $S_1 = \{1, 4\}$ et $S_2 = \{2, 3\}$ forment une partition parfaite $1 + 4 = 2 + 3$.

Définition 2.4 – Partition parfaite d'un sac impair

Un sac d'entiers S est dit *impair* si la somme de tous ses entiers est un nombre impair.

Une *partition parfaite* d'un sac impair est une partition (S_1, S_2) telle que la différence entre la somme des entiers de S_1 et celle de S_2 est égale à 1.

En effet, lorsque la somme totale est impaire, il est impossible de partager les entiers en deux sacs ayant exactement la même somme. La meilleure partition possible est donc celle où les deux sommes diffèrent de 1.

Exemple – Partition parfaite d'un sac impair

Le sac $S = \{1, 2, 3, 4, 5\}$ est impair car $1 + 2 + 3 + 4 + 5 = 15$.

- $S_1 = \{2, 4\}$ et $S_2 = \{1, 3, 5\}$ ne forment pas une partition parfaite, car $2 + 4 = 6$ et $1 + 3 + 5 = 9$.
- $S_1 = \{1, 2, 5\}$ et $S_2 = \{3, 4\}$ forment une partition parfaite, car $1 + 2 + 5 = 8$ et $3 + 4 = 7$, et la différence entre les deux sommes est 1.

Définition 2.5 – Problème de partitionnement

En informatique, le problème de partitionnement consiste à déterminer si une partition parfaite d'un sac d'entiers existe.

Ce problème est connu pour être difficile en informatique théorique, classé *NP-complet* :

- on peut vérifier rapidement si une partition candidate est correcte ;
- mais trouver une partition parfaite pour n'importe quel sac peut être très difficile.

Cependant, plusieurs algorithmes permettent de le résoudre de manière efficace pour des instances de taille raisonnable, soit exactement, soit approximativement. Pour cette raison, on l'appelle parfois :

le plus facile des problèmes difficiles.

Remarque 2.1 Symétries du problème

Ce problème admet une symétrie évidente puisque l'on peut inverser la partition, c'est-à-dire échanger les ensembles S_1 et S_2 .

De manière générale, on peut échanger des objets entre S_1 et S_2 tant que les sommes des deux sacs restent identiques.

Exemple – Symétries du problème

soit le sac $S = \{1, 2, 4, 6, 8, 9\}$ et la partition parfaite de somme égale à 15.

$$S_1 = \{6, 9\}, \quad S_2 = \{1, 2, 4, 8\}.$$

Inversion de la partition On peut inverser la partition et les sommes restent identiques.

$$S_1 = \{1, 2, 4, 8\}, \quad S_2 = \{6, 9\}$$

Échange partiel de nombres On peut aussi échanger des nombres sans changer les sommes, par exemple en remplaçant $6 \in S_1$ par $2, 4 \in S_2$.

$$S_1 = \{2, 4, 9\}, \quad S_2 = \{1, 6, 8\},$$

La symétrie permet de réduire la complexité ; cherchez-la partout.

– Alan J. Perlis

TODO Insérer des médailles (chocolat, bronze, argent, or) pour la difficulté des différentes sections.

3 Partition parfaite des entiers de 1 à n

Exercice 3.1

Trouver une partition parfaite des entiers de 1 à n pour $n = 6, 7, 8, 9$.

Remarque 3.1 Somme des entiers de 1 à n

La somme des entiers de 1 à n vaut

$$1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}.$$

Une anecdote célèbre raconte que le mathématicien allemand Carl Friedrich Gauss aurait découvert cette formule lorsqu'il était encore écolier.

Son professeur lui aurait demandé de calculer la somme des entiers de 1 à 100 afin d'occuper la classe pendant un moment. Mais Gauss remarqua rapidement que l'on pouvait associer les nombres par paires :

$$1 + 100 = 101, \quad 2 + 99 = 101, \quad 3 + 98 = 101, \dots$$

Chaque paire donne la même somme, 101, et il y a 50 paires. La somme vaut donc $50 \times 101 = 5050$.

Ce raisonnement fonctionne pour tout n et conduit à la formule générale $\frac{n(n+1)}{2}$. Cette formule permet notamment de connaître rapidement la somme totale des objets, ce qui sera utile pour déterminer la capacité du sac dans les problèmes de partition.

Exercice 3.2

Trouvez d'autres partitions parfaites en effectuant des échanges partiels d'entiers entre les deux sacs.

Définition 3.1 – Algorithme

Un *algorithme* est une suite d'étapes simples qui permet de résoudre un problème. Le nombre d'étapes peut varier en fonction de la taille des données.

Remarque 3.2

- Plusieurs algorithmes peuvent répondre à un même problème.
- Un algorithme peut répondre à plusieurs problèmes.

Exercice 3.3

Proposer un algorithme pour trouver une partition parfaite des entiers de 1 à n .

Écrire un mauvais programme est facile ; comprendre un bon programme est difficile.
– Alan J. Perlis

4 Algorithme glouton

Jusqu'à présent, nous avons manipulé des sacs d'entiers sans leur donner de sens concret. Pour appliquer les algorithmes, nous allons maintenant considérer chaque entier comme la *taille* d'un objet réel.

Ainsi, chaque entier du sac correspond à un objet que l'on peut placer dans un sac de capacité donnée. Dans les sections suivantes, nous utiliserons donc le terme *objet* tout en gardant à l'esprit qu'il sont seulement caractérisés par un entier, leur taille.

Il n'y a pas de simplicité véritable, que des simplifications.
— Léon-Paul Fargues

Définition 4.1 – Algorithme glouton

Un *algorithme glouton* est un algorithme qui, étape par étape, fait toujours le choix considéré comme le meilleur à ce moment précis (choix *localement optimal*). Dans certains problèmes, cette approche conduit à un optimum global. Dans le cas général, c'est une *heuristique* qui ne garantit pas de trouver l'optimum global.

Algorithme 4.1 – Algorithme glouton

- Déterminer la capacité du sac.
- Trier les objets par ordre décroissant de taille.
- Tant qu'il reste des objets à placer :
 - placer le plus grand objet dans le sac si sa capacité le permet ;
 - Sinon, retirer l'objet ;
 - Si le sac est rempli, arrêter l'algorithme.

Exercice 4.1

Appliquer l'algorithme glouton sur une instance de type 1.

#	Type	Taille	Entiers	Capacité
1	1	6	11, 8, 7, 5, 2, 1	17
2	1	6	16, 11, 10, 8, 4, 1	25
3	1	9	15, 13, 8, 7, 6, 5, 4, 2, 1	30
4	1	9	16, 12, 10, 9, 6, 5, 3, 2, 1	32
5	1	12	16, 15, 13, 12, 9, 8, 6, 5, 4, 3, 2, 1	47

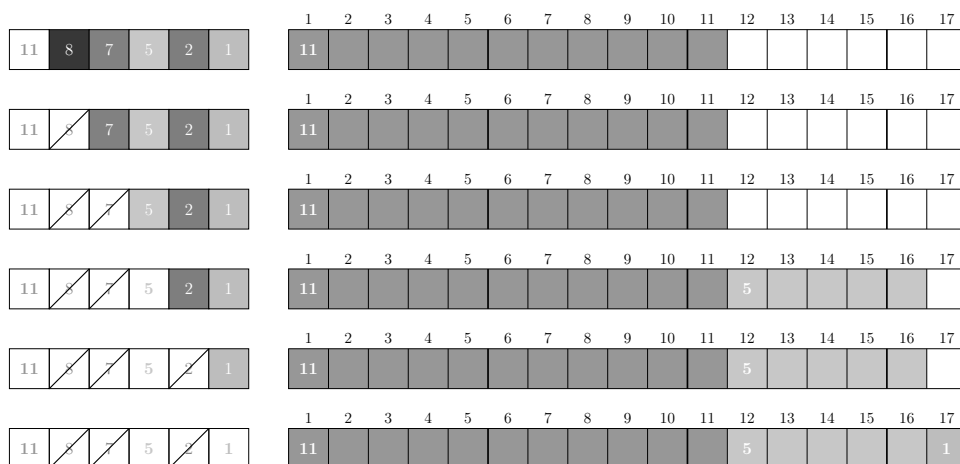


FIGURE 1 – Application de l'algorithme glouton sur l'instance #1 de type 1.

Remarque 4.1 Nombre d'étapes de l'algorithme glouton

Pour un ordre donné des objets, le glouton est très rapide :

- Chaque objet est examiné une seule fois pour être placé dans le sac ;
- Le nombre d'étapes est donc proportionnel au nombre d'objets n .

En pratique, même pour plusieurs centaines d'objet le glouton trouve une partition presque sans transpirer.

Exercice 4.2

Appliquer l'algorithme glouton sur une instance de type 2.

#	Type	Taille	Entiers	Capacité
6	2	6	14, 13, 11, 7, 5, 3	26
7	2	6	13, 12, 11, 10, 7, 4	28
8	2	9	16, 15, 14, 13, 12, 9, 8, 6, 1	47
9	2	9	15, 14, 13, 12, 11, 10, 7, 4, 3	44
10	2	12	16, 15, 13, 12, 11, 10, 8, 7, 6, 4, 3, 2	53

Remarque 4.2 À propos de l'algorithme glouton

- On peut obtenir d'autres solutions en modifiant l'ordre dans lequel les objets sont considérés.
- Des ordres différents donnent parfois la même solution.
- Répéter le glouton avec plusieurs ordres différents permet de générer plusieurs solutions.

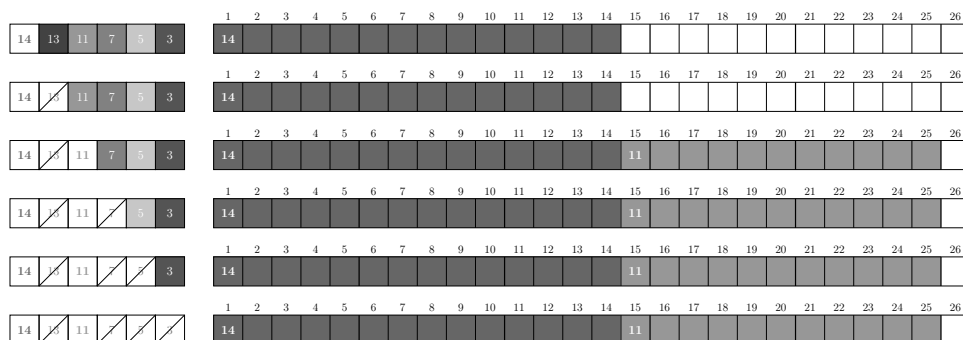


FIGURE 2 – Application de l'algorithme glouton sur l'instance #6 de type 2.

Exercice 4.3A

appliquer l'algorithme glouton pour une partition parfaite des entiers de 1 à n pour $n = 15, 16$.

TODO Insérer les solutions du glouton pour $n = 15, 16$.

Remarque 4.3 Glouton et partition des entiers de 1 à n

L'algorithme glouton est optimal pour le problème de partition des entiers de 1 à n :

- il existe toujours une partition parfaite des entiers de 1 à n ;
- l'algorithme glouton en trouve toujours une.

Preuve : supposons qu'à une étape de l'algorithme on ne puisse pas placer l'entier k dans le sac. Dans ce cas, la capacité restante du sac est égale à la différence entre la capacité totale et la somme des entiers déjà placés dans le sac.

Or les entiers encore disponibles sont exactement $1, 2, \dots, k$. Il reste donc toujours un entier égal à cette capacité restante, que l'on peut placer dans le sac. L'algorithme peut donc toujours compléter le sac.

5 Algorithme glouton répété

Si l'on écrivait des programmes dès l'enfance, on saurait probablement mieux les lire une fois adultes. – Alan J. Perlis

Algorithme 5.1 – Algorithme glouton répété

Répéter tant que le sac n'est pas complètement rempli :

- Appliquer l'algorithme glouton.
- Si le sac est rempli, arrêter l'algorithme.
- Sinon, retirer le plus grand objet du sac restant et recommencer.

Exercice 5.1

Appliquer l'algorithme glouton répété sur une instance de type 2.

#	Type	Taille	Entiers	Capacité
6	2	6	14, 13, 11, 7, 5, 3	26
7	2	6	13, 12, 11, 10, 7, 4	28
8	2	9	16, 15, 14, 13, 12, 9, 8, 6, 1	47
9	2	9	15, 14, 13, 12, 11, 10, 7, 4, 3	44
10	2	12	16, 15, 13, 12, 11, 10, 8, 7, 6, 4, 3, 2	53

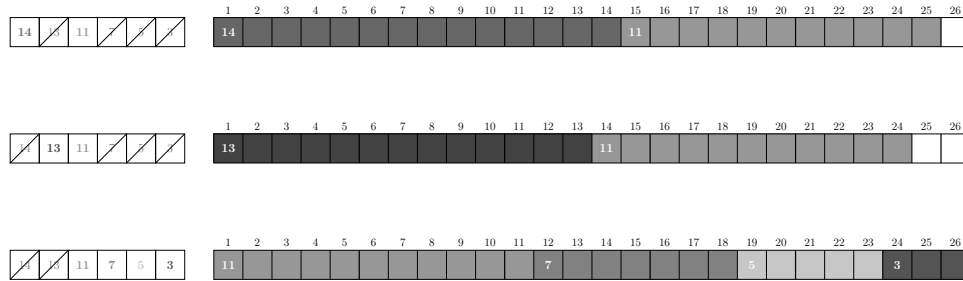


FIGURE 3 – Application de l'algorithme glouton répété sur l'instance #6 de type 2.

Remarque 5.1À propos de l'algorithme glouton répété

- L'algorithme glouton répété examine uniquement des solutions distinctes, car à chaque itération le plus grand objet restant est éliminé avant d'appliquer à nouveau l'algorithme glouton.
- l'algorithme explore au plus n solutions (une par objet éliminé).

Exercice 5.2

Appliquer l'algorithme glouton répété sur une instance de type 3.

#	Type	Taille	Entiers	Capacité
11	3	6	13, 11, 9, 8, 6, 4	25
12	3	6	16, 13, 12, 11, 7, 3	31
13	3	9	16, 15, 14, 10, 9, 8, 6, 5, 3	43
14	3	9	16, 15, 13, 11, 9, 7, 5, 4, 3	41
15	3	12	16, 15, 14, 13, 12, 11, 10, 8, 7, 6, 4, 3	59

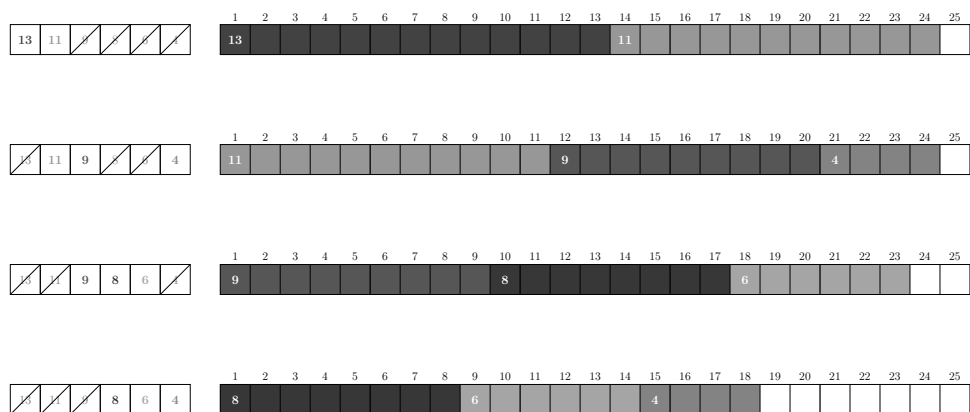


FIGURE 4 – Application de l'algorithme glouton répété sur l'instance #11 de type 3.

6 Programmation dynamique

Pour comprendre un programme, il faut se mettre dans la peau de la machine et du programme.

– Alan J. Perlis

Algorithme 6.1 – Algorithme de programmation dynamique

- Déterminer la capacité du sac.
- Trier les objets par ordre décroissant.
- Répéter tant qu'il reste des objets à placer :
 - Pour chaque case marquée, en partant de la dernière :
 - Calculer la case atteinte en plaçant immédiatement l'objet après la case marquée.
 - Si cette case n'est pas encore marquée, placer un marqueur correspondant à l'objet.
 - Si la dernière case (capacité) est marquée, le sac est rempli et l'algorithme s'arrête.

Algorithme 6.2 – Reconstruction du sac en programmation dynamique

- Tant qu'il reste des marqueurs dans le sac :
- Sélectionner le marqueur le plus à droite.
 - Placer l'objet correspondant à ce marqueur dans le sac en retirant les marqueurs sous l'objet.

Exercice 6.1

Appliquer la programmation dynamique sur une instance de type 3.

#	Type	Taille	Entiers	Capacité
11	3	6	13, 11, 9, 8, 6, 4	25
12	3	6	16, 13, 12, 11, 7, 3	31
13	3	9	16, 15, 14, 10, 9, 8, 6, 5, 3	43
14	3	9	16, 15, 13, 11, 9, 7, 5, 4, 3	41
15	3	12	16, 15, 14, 13, 12, 11, 10, 8, 7, 6, 4, 3	59

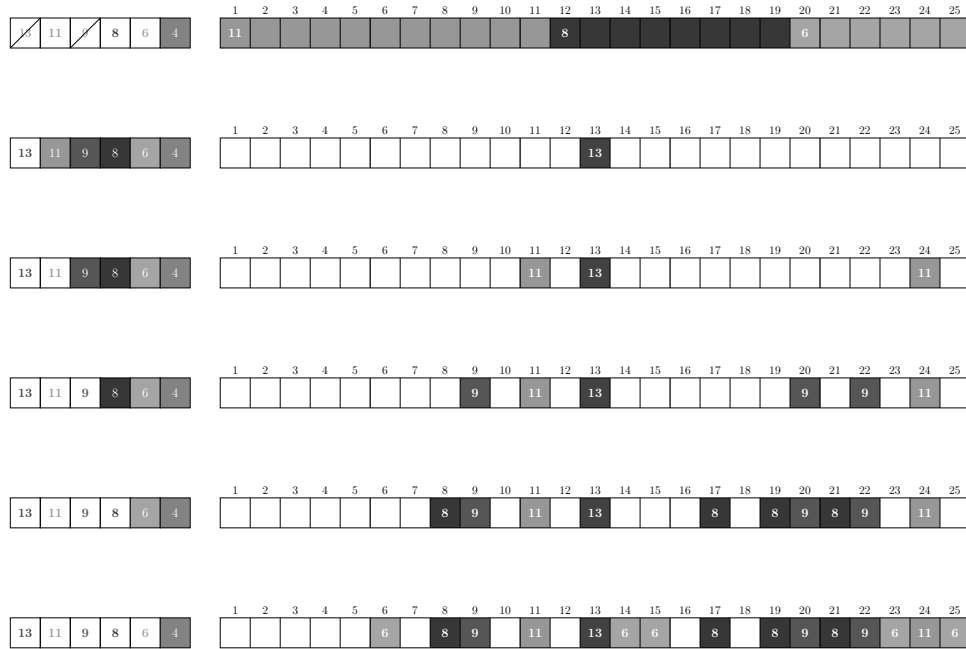


FIGURE 5 – Application de la programmation dynamique sur l'instance #11 de type 3.

Remarque 6.1 Nombre d'étapes de la programmation dynamique

L'algorithme de programmation dynamique est beaucoup plus gourmand en étapes que les gloutons. Pour n objets et une capacité C du sac :

- Il faut examiner toutes les cases marquées pour chaque objet, et il y a au plus C cases marquées ;
- Le nombre d'étapes est donc proportionnel à $n \cdot C$, ce qui est bien plus que le glouton.

Cette complexité ne dépend pas seulement de la taille de l'entrée (le nombre d'objets), mais aussi de leurs valeurs, ici la capacité du sac. Pour des petites valeurs ou une capacité raisonnable, l'algorithme reste rapide, mais pour de grandes valeurs, le temps de calcul peut devenir important.

Exercice 6.2

Appliquer la programmation dynamique sur une instance de type 4.

#	Type	Taille	Entiers	Capacité
16	4	6	16, 15, 11, 4, 2, 1	24
17	4	6	15, 14, 9, 8, 6, 1	26
18	4	8	18, 15, 13, 10, 8, 5, 3, 2	37
19	4	8	18, 15, 13, 10, 8, 5, 3, 2	37
20	4	8	18, 17, 16, 15, 14, 5, 2, 1	44

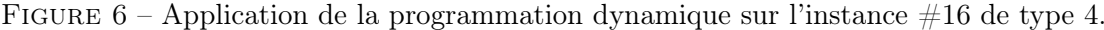
Insérer devise shadok : S'il n'y a pas de solution, c'est qu'il n'y a pas de problème. Zut, les images ne sont pas libres de droit.

Remarque 6.2 Optimalité de la programmation dynamique

L'algorithme de programmation dynamique garantit de trouver la partition la plus équilibrée :

- il trouve une partition parfaite si elle existe ;
- sinon, il minimise la différence entre les sommes des deux sous-sacs.

On sacrifie la rapidité pour être sûr d'obtenir la solution optimale.



7 Tester et Générer

Algorithme 7.1 – Algorithme Tester-et-Générer

- Déterminer la capacité.
- Trier les objets par ordre décroissant.
- Descente : Répéter tant que le dernier objet n'est pas dans le sac :
 - Répéter pour chaque objet :
 - ranger l'objet dans le sac si la capacité le permet.
 - Si le sac est rempli, arrêter l'algorithme.
 - Sinon, effectuer un retour arrière simple : retirer le plus petit objet du sac.
- Retour arrière sauté quand le plus petit objet du deck est dans le sac.
 - Retirer les objets du sac trouver jusqu'à ce que vous trouviez le plus petit objet que vous n'avez pas pu faire rentrer.
 - Sinon éliminer l'objet.

8 Problèmes connexes

En programmation, chaque tâche est un cas particulier d'une idée plus générale – et on s'en rend compte parfois trop vite.

Le problème de partition appartient à une grande famille de problèmes d'optimisation étudiés en informatique et en recherche opérationnelle. De nombreux problèmes célèbres lui sont étroitement liés.

Somme de sous-ensembles on dispose d'un sac d'entiers et d'une capacité donnée. La question est de savoir s'il existe un sous-ensemble d'objets dont la somme est exactement égale à cette capacité.

Le problème de partition en est un cas particulier où la capacité est la moitié de la somme totale des entiers.

Problème du sac à dos chaque objet possède une taille et une valeur. Le but est de choisir des objets à placer dans un sac de capacité limitée de manière à maximiser la valeur totale. Contrairement au problème de partition, il ne s'agit plus de trouver une somme exacte, mais de sélectionner la meilleure combinaison possible en termes de valeurs.

Ordonnancement sur deux machines Un problème très proche apparaît en *ordonnancement*. Supposons que plusieurs tâches doivent être réparties sur deux machines identiques. Chaque tâche a une durée d'exécution. On cherche à minimiser le délai total, c'est-à-dire la date de fin de la dernière tâche. Pour cela, il faut répartir les tâches entre les deux machines de manière à équilibrer au mieux leurs durées totales.

Ce problème revient donc à chercher une partition des durées des tâches aussi équilibrée que possible.

Problème de mise en boîtes On dispose de plusieurs sacs de capacité fixe et d'objets de tailles différentes. Le but est de ranger tous les objets en utilisant le moins de sacs possible. Ce problème généralise l'idée de remplir un sac avec des objets de tailles données.

Ces problèmes [1–5] apparaissent dans de nombreux domaines : logistique, planification de tâches, allocation de ressources ou encore gestion de données. Ils illustrent l'importance des algorithmes pour organiser et optimiser des ressources limitées.

La simplicité ne précède pas la complexité, elle la suit.
– Alan J. Perlis

#	Type	Taille	Entiers	Capacité
1	1	6	11, 8, 7, 5, 2, 1	17
2	1	6	16, 11, 10, 8, 4, 1	25
3	1	9	15, 13, 8, 7, 6, 5, 4, 2, 1	30
4	1	9	16, 12, 10, 9, 6, 5, 3, 2, 1	32
5	1	12	16, 15, 13, 12, 9, 8, 6, 5, 4, 3, 2, 1	47
6	2	6	14, 13, 11, 7, 5, 3	26
7	2	6	13, 12, 11, 10, 7, 4	28
8	2	9	16, 15, 14, 13, 12, 9, 8, 6, 1	47
9	2	9	15, 14, 13, 12, 11, 10, 7, 4, 3	44
10	2	12	16, 15, 13, 12, 11, 10, 8, 7, 6, 4, 3, 2	53
11	3	6	13, 11, 9, 8, 6, 4	25
12	3	6	16, 13, 12, 11, 7, 3	31
13	3	9	16, 15, 14, 10, 9, 8, 6, 5, 3	43
14	3	9	16, 15, 13, 11, 9, 7, 5, 4, 3	41
15	3	12	16, 15, 14, 13, 12, 11, 10, 8, 7, 6, 4, 3	59
16	4	6	16, 15, 11, 4, 2, 1	24
17	4	6	15, 14, 9, 8, 6, 1	26
18	4	8	18, 15, 13, 10, 8, 5, 3, 2	37
19	4	8	18, 15, 13, 10, 8, 5, 3, 2	37
20	4	8	18, 17, 16, 15, 14, 5, 2, 1	44

Références

- [1] Problème de mise en boîtes (bin packing problem). https://fr.wikipedia.org/wiki/Probl%C3%A8me_de_bin_packing.
- [2] Problème de sac-à-dos (knapsack problem). https://fr.wikipedia.org/wiki/Probl%C3%A8me_du_sac_%C3%A0_dos.
- [3] Problème d'ordonnancement avec des machines identiques (identical-machines scheduling). https://en.wikipedia.org/wiki/Identical-machines_scheduling.
- [4] Problème de partition (partition problem). https://fr.wikipedia.org/wiki/Probl%C3%A8me_de_partition.
- [5] Problème de la somme des sous-ensembles (subset sum problem). https://fr.wikipedia.org/wiki/Probl%C3%A8me_de_la_somme_de_sous-ensembles.

- [6] Alan J. Perlis. Special feature : Epigrams on programming. *SIGPLAN Not.*, 17(9) :7–13, September 1982. ISSN 0362-1340. doi : 10.1145/947955.1083808. URL <https://doi.org/10.1145/947955.1083808>.